End-to-End Vertical Slice and Automated Verification

ISE A4 | Intelligent Software Engineering | 8 November 2026

Project context: Assignment Deadline Change Tracking and Verification System (continues A1, A2 and A3)

Purpose and principal result

This assignment delivers one small, complete and reversible slice of user value for the deadline-change workflow. A student sees a detected change to a tracked assignment deadline, sees where it came from, and **confirms** it (the personal tracked deadline moves to a new version with history) or **rejects** it (nothing changes). The confirmed result can be exported as an iCalendar task. This is option B of the A3 ADR-001: a separate verification boundary plus one controlled export adapter.

The slice runs entirely on the author's machine: a Python standard-library HTTP server bound to 127.0.0.1 and a SQLite file. Nothing is deployed. The unchanged A2 domain model is reused. The A2 invariants are enforced in the server and, for ownership, versioning and state transitions, again as SQLite triggers. Every refusal returns a code, a message and a recovery action. Negative results are retained, including a failed first test run that exposed a real defect.

Slice identity and environment

Item	Recorded identity
Repository	ISE-A4 study repository; submitted at tag `ise-a4-v1.0`
Commits	`6625c5d` A2 baseline (byte-identical); `8de033c` A4 slice; documentation and evidence commits follow
Runtime	CPython 3.12.4; SQLite 3.45.3; Windows 11 (10.0.22631); tzdata 2023.3
Dependencies	Python standard library only (`sqlite3`, `http.server`, `unittest`, `zoneinfo`); `tzdata` on Windows
Data	`a4-synthetic-fixtures-v1`, SHA-256 b39689db3068…d8ad; derived from the A2 fixture IDs and dates
Determinism	Fixed test tokens and idempotency keys; no random inputs; LF line endings enforced by .gitattributes

Recorded outcomes

Evidence	Outcome	Interpretation
Automated checks	72 checks, all pass	12 unit, 30 integration, 12 end-to-end, 18 inherited A2 checks (1 expected failure kept)
First test run	2 of 71 failed	One product defect, one faulty test oracle; both retained and fixed
Repeat runs	10 of 10 OK	No flaky results from the threaded concurrency tests in this environment
Error behaviour	13 scenarios captured	Responses recorded from a real local server, not hand-written
Rollback drill	Levels 1–5 PASS	Feature flag, compensating revert, restore, schema down, code rollback
Peer review	PENDING	Protocol in PEER-REVIEW.md; insert result before submission
User task test	PENDING	Protocol in USER-TASK-TEST.md; insert result before submission

All records are synthetic. No real student data, university credentials, live LMS or network deployment was used. A1 simulated pilot data remain simulated.

Task contract and requirement link

The slice was selected because it is the smallest change that delivers the A1 user value end to end and closes the A3 risk R1. The full contract, with every acceptance criterion mapped to named tests, is in TASK-CONTRACT.md.

Source	Requirement	Taken into the slice
A1	Normal scenario: a deadline changes and the student must update their own record	Tracked deadline changes only after the owner confirms; source is kept
A1	Abuse scenario: an untrusted message claims a new date	Unverified sources cannot be applied (422) but can be rejected
A2	I1 owner-only review; I2 verified provenance; I3 versioned history, no lost update; I4 rejection never mutates; I5 explicit zones	Enforced in `service.py`; I1, I3 and I4 also as SQLite triggers
A2	Counterexample W6: direct mutation bypasses the in-memory rules	Re-tested on SQLite; blocked by triggers (TestStorageGuards)
A3	R1: direct writes lack verification and stale-version protection; ADR-001 option B	Confirmation endpoint and one VTODO `.ics` export adapter

User story and scope

As a student who tracks my own deadlines, when a possible deadline change has been detected, I want to see where it came from and confirm or reject it, so that my record changes only when I agree and I can export the result to my calendar.

Non-goals: detecting changes from a real LMS or email (candidates are seeded fixtures), institutional authentication, deployment, reading calendar edits back, multi-device sync, and the deletion workflow (A2 retention rules stay design requirements).

Acceptance criteria

AC	Criterion	Evidence
1	Owner confirms a verified pending change: 200, version +1, one history row with actor and source	Integration normal path; E2E journey
2	Reject: 200, deadline, version and history unchanged	Integration; regression test for outdated reject
3	Duplicates: same key replays; same key with a different body 422; concurrent clicks apply once	TestDuplicates (8 threads); E2E double-click
4	Stale: wrong expected_version 409; outdated candidate 409; conflicting race has one winner	TestStaleVersions; E2E stale page
5	Unauthorized: 401 without valid token; 404 for other students' data; actor never from body	TestAuthentication; E2E TestUnauthorized
6	Abuse: unverified source cannot be applied	Integration abuse tests
7	Boundaries: 413, 415, 400, 422 offset mismatch, 404/405 routes, `.ics` escaping	Unit tests; E2E TestBoundaries
8	A mid-transaction fault leaves no partial state and the action succeeds afterwards	Fault-injection test; error-behaviour capture
9	Rollback works at every level	TestRollback; TestFeatureFlagOff; rollback drill
10	Representative users complete confirm and stale-recovery tasks	Manual: USER-TASK-TEST.md, pending

Implementation and change boundary

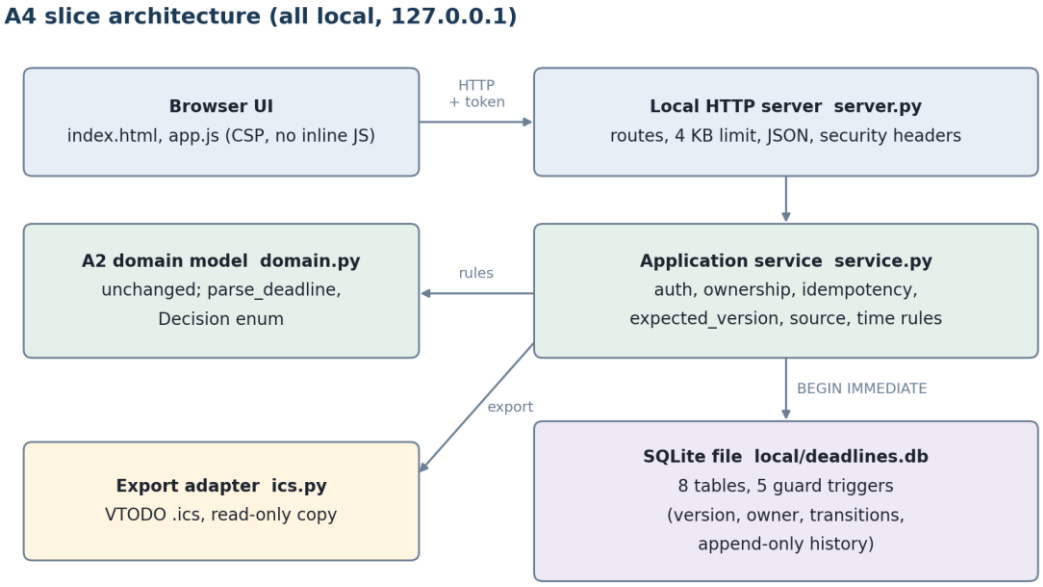


Figure 1. Slice components. Every rule runs in the server process; the browser only displays state and messages. The official LMS deadline stays outside this system.

File	Lines	Role
migrations/001_up.sql / _down.sql	124 / 15	8 tables, 5 guard triggers; reversible schema
src/service.py	289	Authentication, ownership, idempotency, version checks, decision transaction, compensating revert
src/server.py	228	Local JSON API, body limit, content-type checks, security headers, feature flag, structured log
src/static/ (html, js, css)	249	Sign-in, assignment list, confirm/reject, error banner with Retry and Reload latest, export
src/db.py, ics.py, fixtures.py	155	Migrations, backup/restore, RFC 5545 export, fixture loading with hashed tokens
src/domain.py (A2)	0 changed	Reused unchanged: `parse_deadline`, `Decision`

The slice commit adds 2,180 lines: about 1,060 product lines (code, SQL and UI), 623 test lines and 455 lines of reproduction scripts. No existing line is changed, so a reviewer can read the slice against the baseline with `git diff 6625c5d 8de033c`. A reviewer may still find the commit large for a "small diff". Most of it is tests and SQL guards, which were judged proportional to the write-path risk.

Write path: authorization, duplicates and stale versions

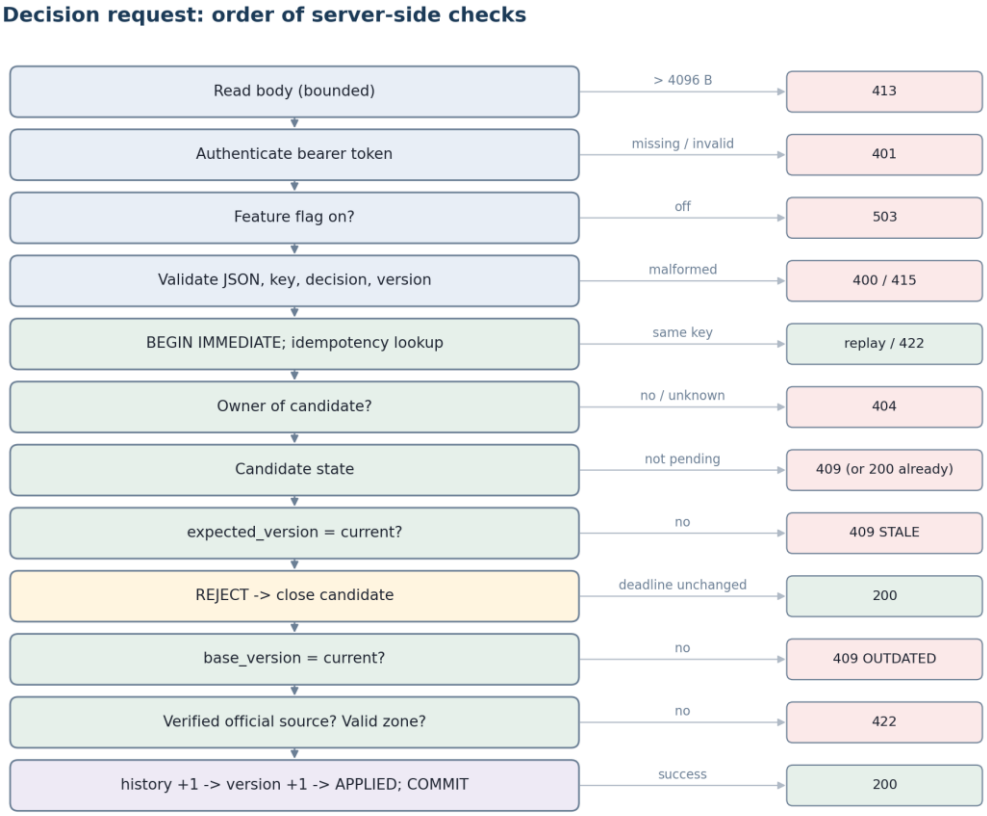


Figure 2. Order of server-side checks for one decision request. REJECT is evaluated before the outdated-candidate check so an outdated change can always be rejected (the first test run found this ordering wrong).

Concern	Mechanism	Evidence
Server authorization	Bearer token hashed (SHA-256) and looked up; student taken only from the token; body shape is exact, so it cannot name an actor	401 tests; actor-spoof E2E test; no plaintext tokens in DB dump
Other students' data	Ownership check returns 404 for both "not yours" and "does not exist"; 401/404 outcomes are never cached under an idempotency key	Cross-student tests: no state change, zero idempotency rows
Duplicate requests	`Idempotency-Key` per student; stored response replayed with header `Idempotent-Replayed`; different body with the same key gives 422	8-thread and 5-click tests: exactly one history row
Stale versions	`expected_version` from the page compared inside `BEGIN IMMEDIATE`; candidate `base_version` must equal the current version	Concurrent conflicting confirms: one 200, one 409
Atomic state change	History row, then version +1 (trigger requires the matching history row), then candidate APPLIED, then idempotency row, then COMMIT	Injected fault: version, history, candidate and key all unchanged
Boundaries	4 KB body cap read before authentication (keeps the connection usable), JSON content type, ID charset, offset must match the named zone	Boundary and keep-alive E2E tests

Automated verification

Checks are proportional to risk: the write path and storage guards carry the most integration tests; the HTTP layer is exercised end to end; pure functions have unit tests. One command runs everything and keeps the raw output with an environment header:

```
python scripts/run_tests.py
python scripts/capture_error_behaviour.py
python scripts/rollback_drill.py
```

Level	File	Checks	What it exercises
Unit	tests/test_unit.py	12	Request validation bounds, `.ics` UTC conversion and injection escaping, flag parsing
Integration	tests/test_integration.py	30	Service on a real SQLite file: auth, duplicates, threads, stale versions, abuse, triggers, fault injection, rollback
End to end	tests/test_e2e.py	12	Real HTTP server on an ephemeral port: user journey, 401/404, 413/415/400/405, keep-alive, feature flag off
Regression	tests/test_domain.py	18	Unchanged A2 suite, including its retained expected failure

Run history (all retained in evidence/test-runs)

Run	Code state	Result	Note
first-run	uncommitted	FAIL 2 / 71	Defects below
after-fix	uncommitted	PASS 72	One regression test added
clean-commit	`8de033c`	PASS 72	Header shows git_dirty=yes because documentation was not yet committed
10 repeated runs	`8de033c`	10 / 10 OK	Concurrency tests stable on this machine

Defects found by the checks

Product defect (found by the E2E journey). When a change was outdated, the error message advised "Reject it", but the server checked the outdated condition before the reject path and refused the rejection, so a user could get stuck. Rejection never mutates the deadline, so the check now applies to confirmation only. A regression test was added.

Test-oracle defect. The `.ics` injection test counted the substring `END:VTODO`. The escaped title legitimately contains it on the same line. The oracle now counts whole lines. The code was correct.

Found in self-review before testing. The first POST handler authenticated before reading the body. On a keep-alive connection the unread body would corrupt the next request. The body is now read (bounded) first, and a dedicated E2E test covers it.

What the tests do not show

The E2E tests drive the same HTTP API the page uses, but not a real browser, so `app.js` rendering and the banner flow are verified only by manual use and the user task test. Concurrency was tested with threads against one SQLite file, not with several processes or machines. No coverage percentage is claimed.

Usability and error-behaviour evidence

The table is an excerpt of evidence/error-behaviour.md, generated by running each scenario against a real local server. The message and recovery columns are exactly what the UI banner shows.

Scenario	HTTP	Message shown	Recovery offered
Confirm official change	200	Deadline updated.	Now version 2
Retry with the same key	200	Same response, marked replayed	None needed
Second tab, new key	200	Already applied. Nothing was changed twice.	None needed
Stale page (v1 shown, v2 current)	409	Your page showed version 1, but the deadline is now version 2.	Reload latest, then decide again
Outdated candidate	409	Detected against an older deadline version.	Reject it (works since the fix)
Unverified chat source	422	Unverified source; cannot be applied.	Check the official course page; Reject
No token	401	Sign in with your access token.	Paste token and sign in
Other student's change	404	Change not found.	Reload your list
Body over 4 KB	413	Request body exceeds 4096 bytes.	-
Feature flag off	503	Temporarily switched off. Deadline unchanged.	View and export still work

Peer review

PENDING. Protocol and checklist are fixed in PEER-REVIEW.md: reviewer, date, commit reviewed, each comment with severity, author response and resolution. Replace this paragraph with the real outcome (number of comments, what changed, any disagreement kept open).

Representative-user task test

Protocol frozen in USER-TASK-TEST.md: 2–3 students who track their own deadlines, consent read aloud, no personal data, six tasks (T1 read deadline, T2 confirm official change, T3 handle outdated change, T4 handle unverified change, T5 export, T6 recover from a stale page), think-aloud, 5-minute stop rule.

PENDING. Replace with the real results: participants, success per task, times, assists, observed problems, quotes, and any change made in response. Report the actual number of participants, even if lower than planned.

Rollback design and drill

Rollback is layered so the least destructive action is tried first. Each level was executed by `scripts/rollback_drill.py` on a throwaway database; the transcript (tokens redacted) is evidence/rollback-drill.txt.

Level	Mechanism	Data loss	Drill result
1 Feature flag	`A4_DECISIONS_ENABLED=0`: writes return 503, reads and export continue, UI disables buttons	None	Health shows decisions_enabled false
2 Compensating revert	`rollback.py revert-change`: old deadline written back as a new version with a REVERT history row	None; audit kept	v2 to v3 restored; second revert refused (409)
3 Restore backup	SQLite backup API copy taken before the change, restored with the server stopped	Writes after backup	Version 1, 0 history rows
4 Schema down	`001_down.sql`; refuses without `--yes`; automatic safety backup	All slice data (backed up)	Refusal exit 2; schema version 0
5 Code rollback	`git revert` of the slice commit; target checked by extracting HEAD~1 and running its tests	Per levels 1–4	Baseline 6625c5d: OK, 1 expected failure

A revert is refused if later changes exist, because rolling back an earlier change would silently discard newer work; the newest change must be reverted first. Exported `.ics` copies cannot be recalled. SEQUENCE increases with every version, so calendars that honour it replace older copies after re-export.

Limits, residual risk and AI use

What the evidence supports

On the recorded Windows environment, the slice confirms, rejects and exports a tracked deadline. It enforces ownership, idempotency, stale-version, provenance and time-zone rules on the server and in storage, and gives an actionable message for every refusal tested. Every rollback level was executed. This establishes a reviewable local slice, not production readiness.

Limits and residual risk

- Synthetic data only. Change detection is not implemented; candidates are seeded fixtures. The `authority_verified` flag is a fixture assumption, as in A2.
- Demo bearer tokens have no expiry, login flow or TLS. They are acceptable only on 127.0.0.1.
- E2E tests are HTTP-level, not browser-level. Usability depends on the pending user task test.
- Concurrency was tested with threads and SQLite's single-writer lock on one machine. Multi-process load was not tested.
- Anyone with direct access to the SQLite file can drop the triggers. The triggers protect against application bugs, not against the file owner.
- Linux and macOS were not executed. The code uses only portable standard-library modules, but this is untested.

Provenance and AI use

Claude Code (Anthropic, Claude Opus 5.5) assisted with slice selection, code, tests, scripts, figures and drafting; all commands were executed on the author's machine. AI-USE-AND-VERIFICATION.md records accepted and rejected advice. Rejected: hosted deployment, extra frameworks (Flask, FastAPI, Playwright), Python-only invariant enforcement, and 403 for other students' data. Three AI-introduced defects were caught by the tests or self-review and are retained above. No peer review or user-test result was generated by AI.